



Road Crossy

Developing a terminal-based C++ game where a player must cross multiple lanes of obstacles.

Abillon, Ancel, Alvarez, Eborde, Ramos



The Rise of Crossy Road



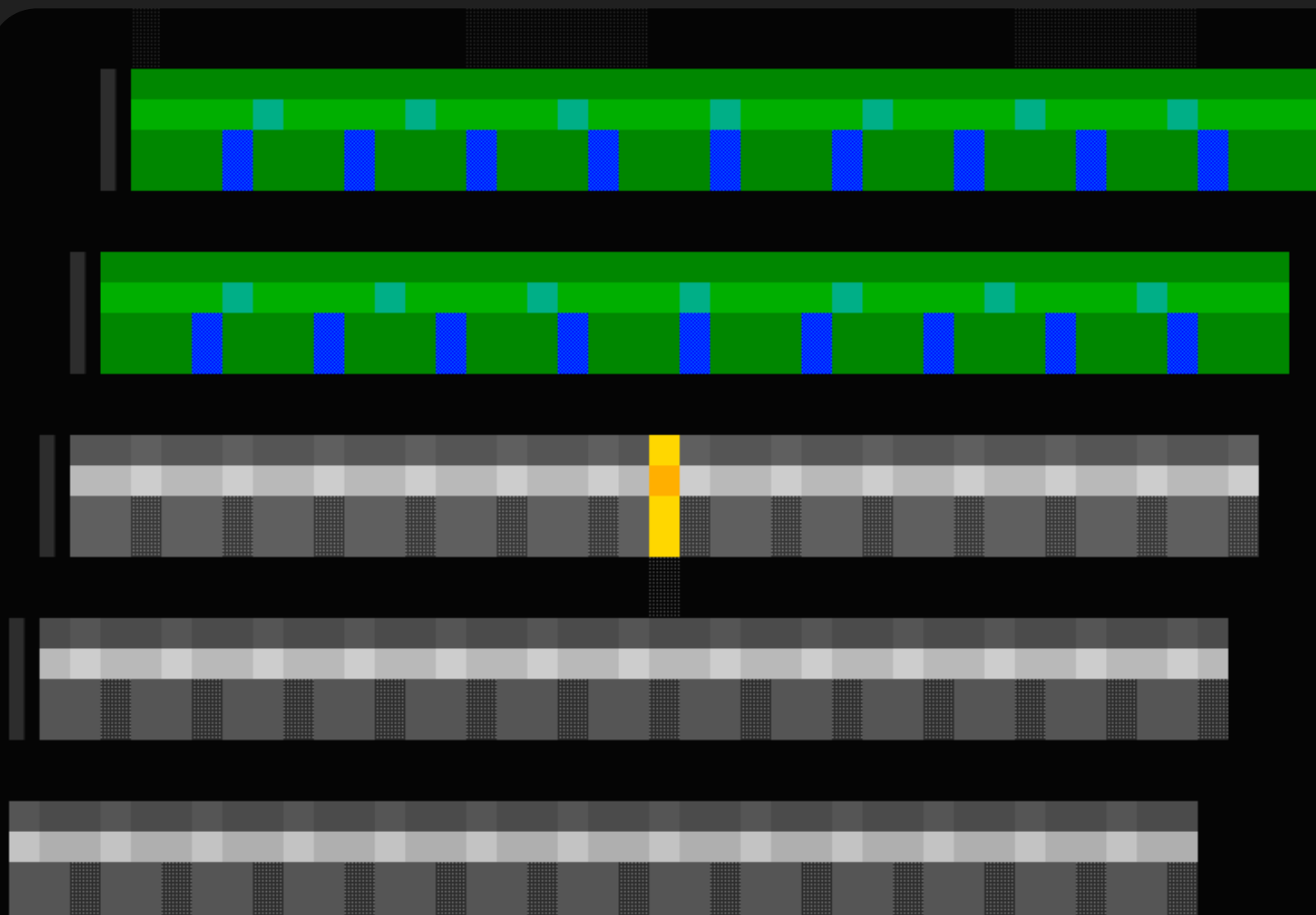
```
ursor and screen control
clear()      { std::cout << "\033[2J\033[H"; }
home()       { std::cout << "\033[H";       }
hide_cursor() { std::cout << "\033[?25l";    }
show_cursor() { std::cout << "\033[?25h";    }
reset_attr() { std::cout << "\033[0m";      }
bold()       { std::cout << "\033[1m";      }
fg(int n)    { std::cout << "\033[38;5;" << n << "m"; }
bg(int n)    { std::cout << "\033[48;5;" << n << "m"; }
move(int r, int c) { std::cout << "\033[" << r << ";" << c << "H"; }

tring-returning ANSI helpers (for building buffered output)
:string sfg(int n) { return "\033[38;5;" + std::to_string(n) + "m"; }
:string sbg(int n) { return "\033[48;5;" + std::to_string(n) + "m"; }
:string sreset()   { return "\033[0m"; }
:string sbold()    { return "\033[1m"; }

on-blocking key read; returns special codes for arrow keys
read_key() {
    unsigned char buf[8] = {};
    int n = (int)read(STDIN_FILENO, buf, sizeof(buf));
    if (n <= 0) return 0;
}
```

PROJECT OVERVIEW

- Terminal-based game
- Simulates 3D using ASCII
- Infinite scrolling gameplay
- Leaderboards



Features & Functionality



INFINITE PROCEDURAL WORLD

Lanes are generated on demand and garbage-collected behind the player.

FOUR TERRAIN TYPES

Safe (cobblestone), Grass, Road (cars), and Water (logs).

PHYSICS

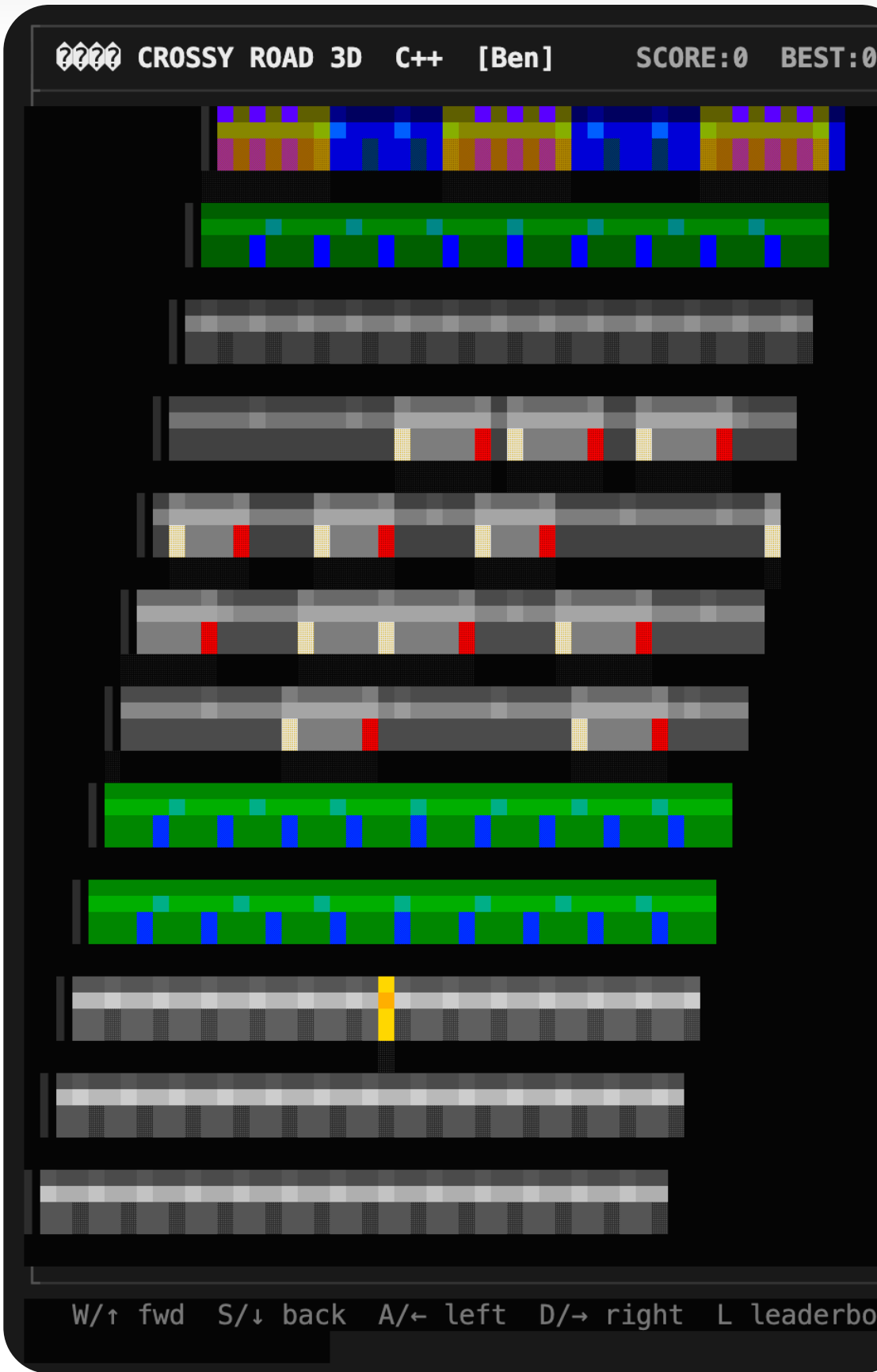
Cars kill on contact. Falling off a log into water is fatal. Logs carry the player downstream.

SCORING

Equals the highest absolute lane index reached in a session.

JUMP ANIMATION

The player character flashes when moving.



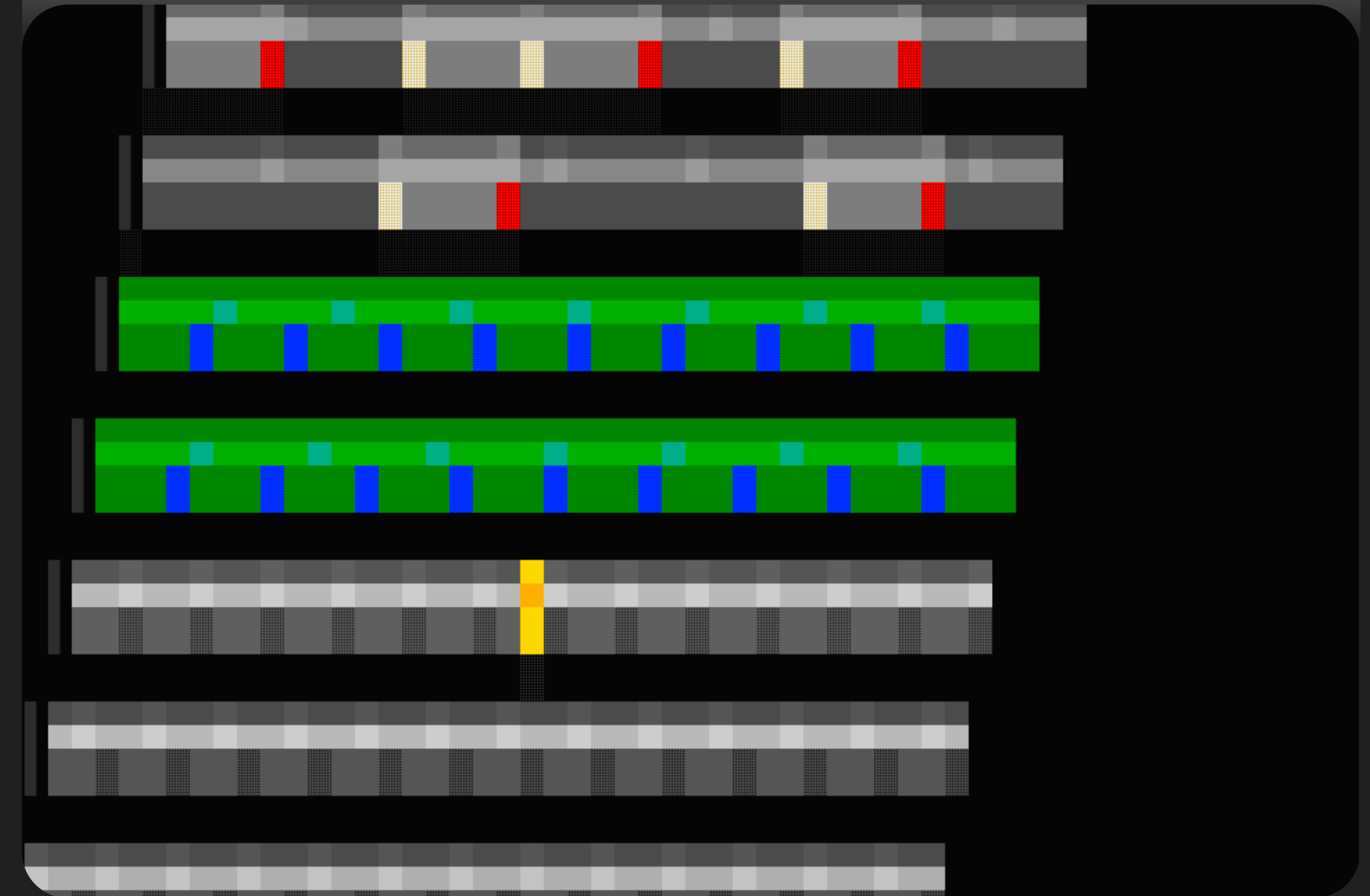
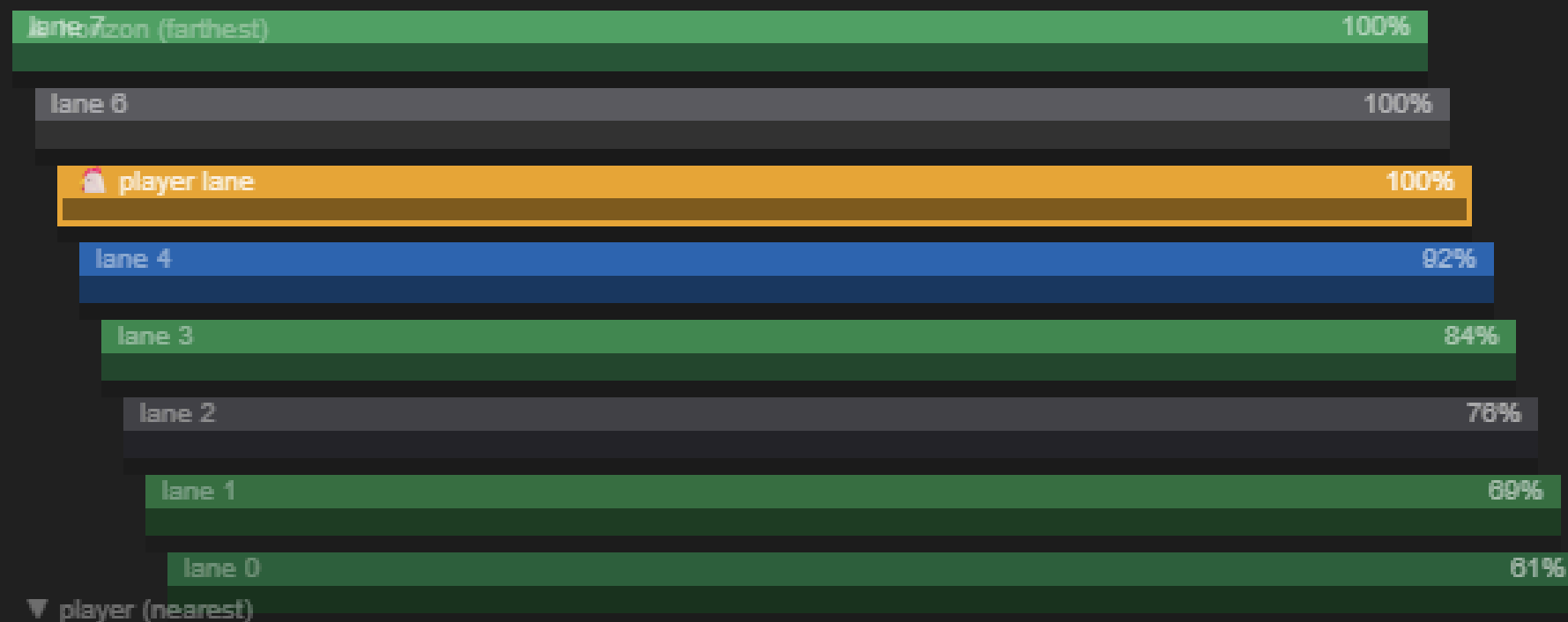
3D Isometric Renderer

EACH LANE RENDERS AS 3 TERMINAL ROWS

Top face (▬), front face (■),
and a drop shadow (▩) —
using Unicode half-block
characters.

ISOMETRIC PERSPECTIVE

Lanes shift one column right
per depth step, producing a
convincing fake-3D look
without a real 3D engine.



DEPTH-AWARE SHADING

Colors darken smoothly with
distance using a fade factor
(100% at player level → ~45%
at the horizon).

FULL DOUBLE-BUFFER

The entire screen is built in
memory and flushed atomically
to minimize flicker.



Leaderboard System

Scores are saved to leaderboard.csv in the working directory after every session.

The file stores one entry per line in the format: name, score.

Entries are sorted in descending order by score; only the top 10 are retained.

The leaderboard displays on the splash screen, can be viewed in-game (L key), and shows again after the session ends.

Rank is calculated and shown in the end-of-game summary box.

🏆 LEADERBOARD 🏆		
RANK	PLAYER	SCORE
1	Mike	70
2	a	57
3	karl	28
4	Karl	25
5	BingChilling	20
6	Bennnnn	9
7	Zen	9
8	Anonymous	6
9	Anonymous	5
10	kent	4

Enter your name:

Game Summary	
Player	: kwent
Score	: 20
Best	: 20
Max Streak	: 20
Moves	: 20
Speed Tier	: Normal
Rank	: #1 on leaderboard

🏆 LEADERBOARD 🏆		
RANK	PLAYER	SCORE
1	kwent	20

CONTROLS	
W / ↑	Move forward (into the 3D dep
S / ↓	Move back
A / ←	Move left
D / →	Move right
L	View leaderboard (in-game or a
R	Restart after death
Q	Quit

🏆 LEADERBOARD 🏆		
RANK	PLAYER	SCORE
No scores yet – be the first!		

Enter your name:


```
// Cursor and screen control
void clear()      { std::cout << "\033[2J\033[H";
void home()      { std::cout << "\033[H";
void hide_cursor() { std::cout << "\033[?25l";
void show_cursor() { std::cout << "\033[?25h";
void reset_attr() { std::cout << "\033[0m";
void bold()      { std::cout << "\033[1m";
void fg(int n)   { std::cout << "\033[38;5;" + to_string(n) + "m";
void bg(int n)   { std::cout << "\033[48;5;" + to_string(n) + "m";
void move(int r, int c) { std::cout << "\033[" + to_string(r) + ";" + to_string(c) + "H";

// String-returning ANSI helpers (for building lib)
std::string sfg(int n) { return "\033[38;5;" + to_string(n) + "m";
std::string sbg(int n) { return "\033[48;5;" + to_string(n) + "m";
std::string sreset()  { return "\033[0m"; }
std::string sbold()   { return "\033[1m"; }

// Non-blocking key read; returns special codes
int read_key() {
    unsigned char buf[8] = {};
    int n = (int)read(STDIN_FILENO, buf, sizeof(buf));
    if (n <= 0) return 0;
    if (buf[0] < 0x20) return 0;
    if (buf[0] < 0x40) return 0;
    if (buf[0] < 0x60) return 0;
    if (buf[0] < 0x7f) return 0;
    if (buf[0] < 0x80) return 0;
    if (buf[0] < 0x90) return 0;
    if (buf[0] < 0xa0) return 0;
    if (buf[0] < 0xb0) return 0;
    if (buf[0] < 0xc0) return 0;
    if (buf[0] < 0xd0) return 0;
    if (buf[0] < 0xe0) return 0;
    if (buf[0] < 0xf0) return 0;
    if (buf[0] < 0xff) return 0;
    return 0;
}
```

Pre Requisites

Installation & Setup

OPERATING SYSTEM

Linux or macOS (POSIX terminal required)

COMPILER

g++ 7+ or clang++ 5+ with C++17 support

TERMINAL

Any 256-color terminal (xterm, iTerm2, GNOME Terminal, etc.)

TERMINAL SIZE

Minimum 80 columns × 45 rows recommended

DISK SPACE

< 1 MB (source + binary + CSV)



Code Structure

Overview

Section	Name	Responsibility
1	Terminal Helpers	Raw mode, cursor control, ANSI color sequences
2	Game Constants	Board dimensions, timing values, and leaderboard settings
3	Leaderboard System	load/save CSV, submitScore, displayLeaderboard
4	Visual Palette	Depth-aware color tables per lane type
5	Linked List	Generic SLinkedList<T> template for obstacles
6	Obstacle	Car/log data: position, width, speed, visual type
7	Lane	Terrain strip with obstacle list; doubly-linked
8	World	Doubly-linked lane list; O(n) laneAt lookup
9	RNG Helpers	Integer and floating-point random helpers
10	Lane Factory	makeLane(), buildInitialWorld(), ensureLanes()

Section	Name	Responsibility
11	Physics	tickLane(), findObs(), physTick(), tryMove()
12	Player	Position, score, animation state
13	3D Render Engine	ScreenBuf, TileInfo, drawLane3D()
14	HUD & Footer	drawHUD3D(), drawFooter3D(), drawDepthRail()
15	Full Frame Render	render3D() – assembles and flushes one frame
16	Input Handling	pollKey(), Key enum
17	Splash Screen	splash() – title art, controls, name prompt
18	Game Loop	runGame() – main event/physics/render loop
19	Entry Point	main(), signal handler cleanup()

Physics Model

TICK_MS = 90 MS

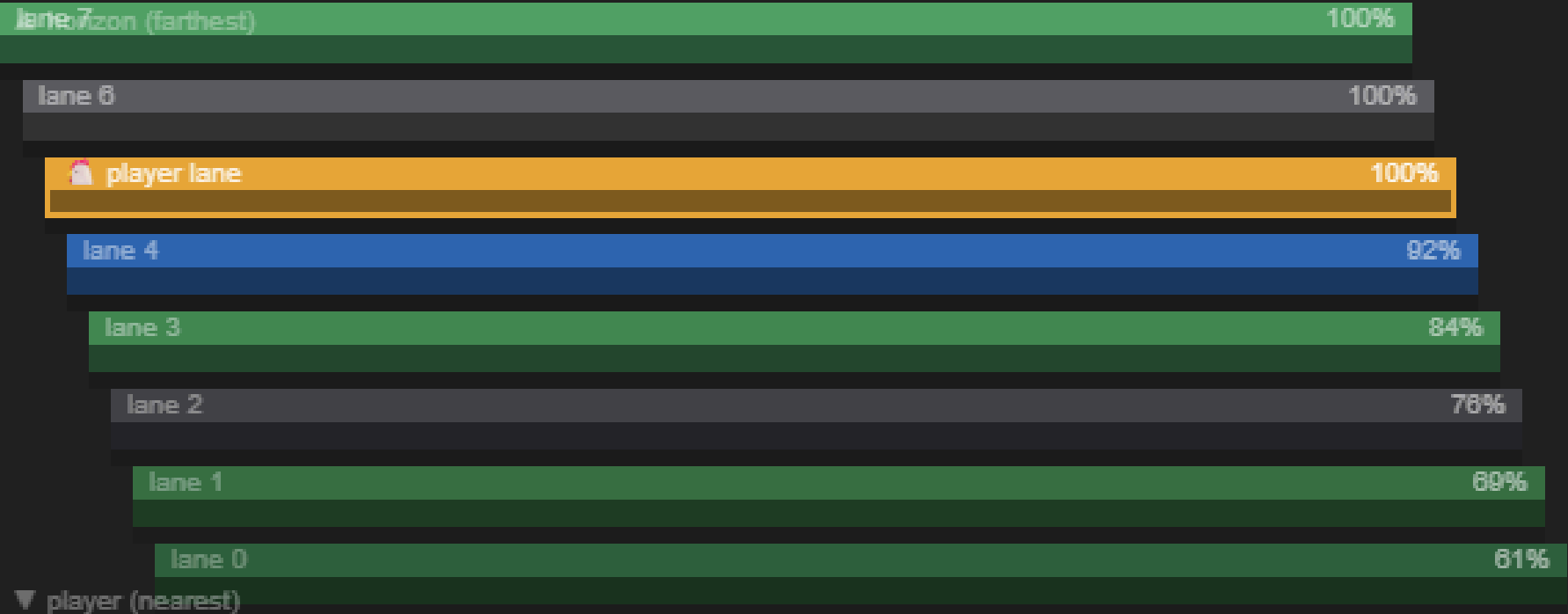
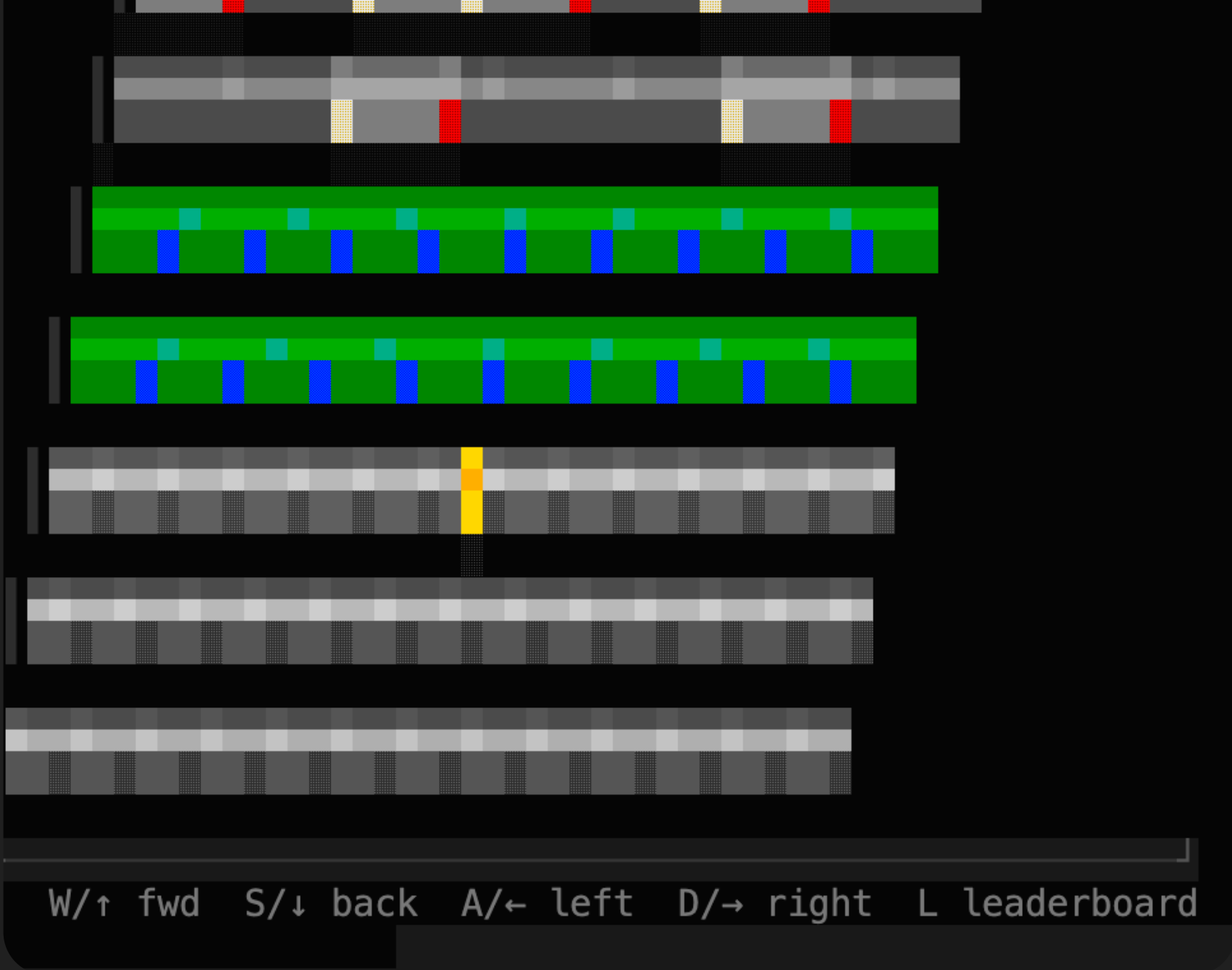
All obstacles in nearby lanes advance by their speed. Log drift is applied to the player if on water.

FRAME_MS = 50 MS

render3D() is called to flush a new frame. ~20 FPS.

INPUT POLL

Checked every loop iteration (~8 ms sleep). Movement is applied immediately on keypress.



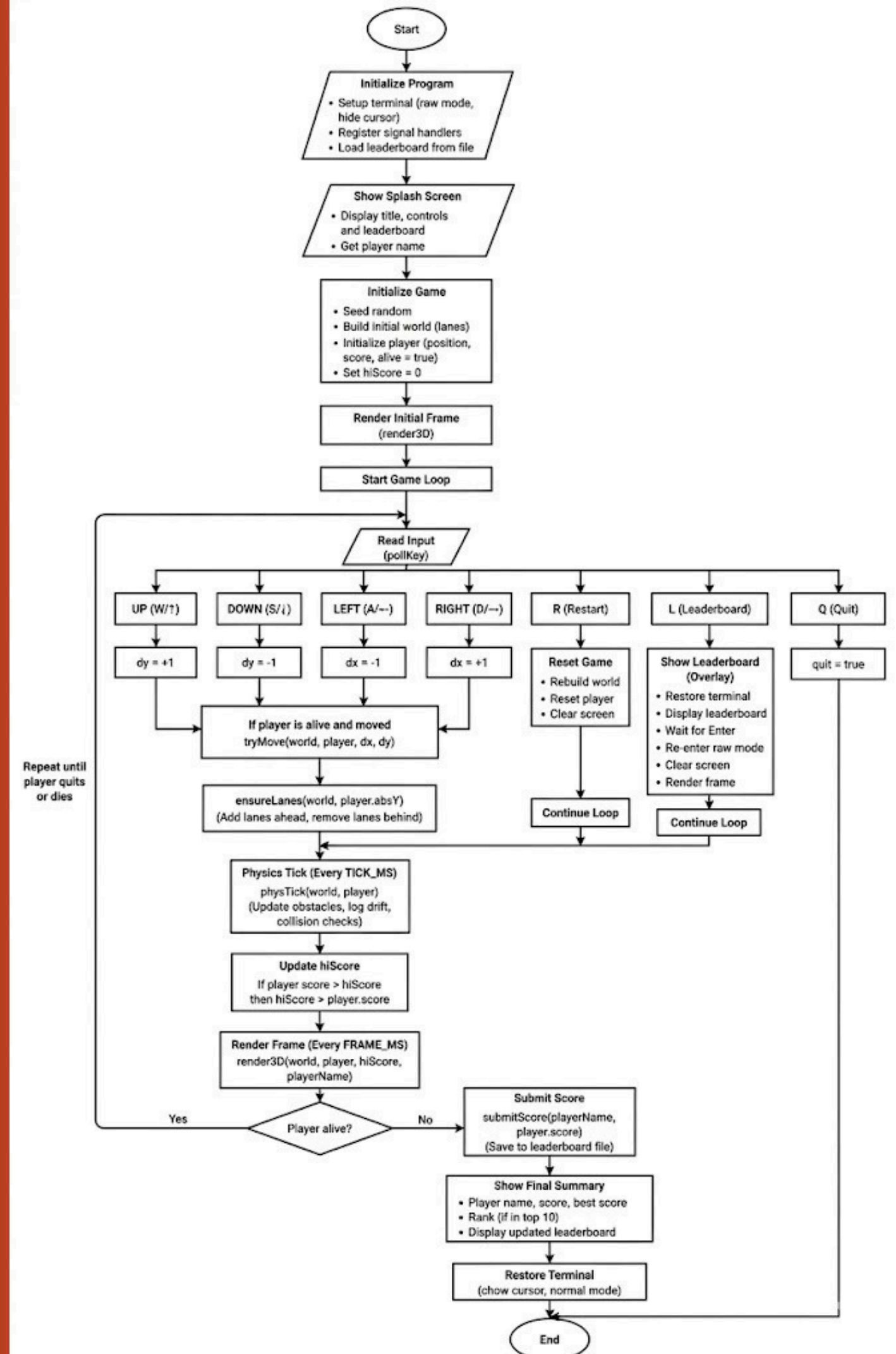
Configuration

Constant	Default	Description
BW	40	Board width in tiles
BH	12	Number of visible lanes
LANE_H	3	Terminal rows per lane
ISO_SKEW	1	Isometric column shift per depth step
PLAYER_LANE	BH-3	Player's fixed screen slot (0=top)
AHEAD_BUFFER	BH+4	Lanes are pre-generated ahead of the player
BELOW_BUFFER	BH+4	Lanes are retained behind the player
TICK_MS	90	Physics update interval (ms)
FRAME_MS	50	Render interval (ms, ~20 fps)
LEADERBOARD_FILE	leaderboard.csv	CSV file path for scores
MAX_LEADERBOARD_ENTRIES	10	Maximum entries retained in the leaderboard

Reference



Flow Chart





DeMo

Thank
You